УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 8

РАЗРАБОТКА ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ СОЗДАНИЯ СЛОВАРЕЙ И ВЫПОЛНЕНИЯ ОПЕРАЦИЙ НАД НИМИ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Разработка простейших программ для создания словарей и выполнения операций над ними

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Словари в Python

Словари — это наборы объектов, доступ к которым осуществляется не по индексу, а по ключу. В качестве ключа можно указать неизменяемый объект, например: число, строку или кортеж. Элементы словаря могут содержать объекты произвольного типа данных и иметь неограниченную степень вложенности. Следует также заметить, что элементы в словарях располагаются в произвольном порядке. Чтобы получить элемент, необходимо указать ключ, который использовался при сохранении значения.

Словари относятся к отображениям, а не к последовательностям. По этой причине функции, предназначенные для работы с последовательностями, а также операции извлечения среза, конкатенации, повторения и др., к словарям не применимы. Равно как и списки, словари относятся к изменяемым типам данных. Иными словами, можно не только получить значение по ключу, но и изменить его.

4.2 Создание словаря в Python

Создать словарь можно следующими способами:
– с помощью функции dict (). **Форматы функции:**
dict(<Ключ1>=<Значение1>[, ..., <КлючN>=<ЗначениеN>])
dict(<Словарь>)
dict(<Список кортежей с двумя элементами (Ключ, Значение)>)
dict(<Список списков с двумя элементами [Ключ, Значение]>)
Если параметры не указаны, то создается пустой словарь.
Примеры:

```
d = dict ()
```

```
print(d) # Создаем пустой словарь
{}
d = dict(a=1, b=2)
print(d)
{'a': 1, 'b': 2}
d = dict ({"a": 1, "b": 2})
print(d) #  Словарь
{'a': 1, 'b': 2}
d = dict ([("a", 1), ("b", 2)])
print(d) #  Список кортежей
{'a': 1, 'b': 2}
d = dict ( [ ["a", 1], ["b", 2]])
print(d) #  Список списков
{'a': 1, 'b': 2}
```

Объединить два списка в список кортежей позволяет функция zip ( ):

```
d = dict ()
print(d) # Создаем пустой словарь
{}
d = dict(a=1, b=2)
print(d)
{'a': 1, 'b': 2}
d = dict ({"a": 1, "b": 2})
print(d) #  Словарь
{'a': 1, 'b': 2}
d = dict ([("a", 1), ("b", 2)])
print(d) #  Список кортежей
{'a': 1, 'b': 2}
d = dict ( [ ["a", 1], ["b", 2]])
print(d) #  Список списков
{'a': 1, 'b': 2}
```

– указав все элементы словаря внутри фигурных скобок. Это наиболее часто используемый способ создания словаря. Между ключом и значением указывается двоеточие, а пары «ключ/значение» записываются через запятую.

Пример:

```
d = dict ()
print(d) # Создание пустого словаря
{}
d = {"a": 1, "b": 2}
print(d)
{'a': 1, 'b': 2}
```

– заполнив словарь поэлементно. В этом случае ключ указывается внутри квадратных скобок:

```
d = {} #  Создаем пустой словарь
d["a"] = 1 # Добавляем элемент 1   (ключ "a")
d["b"] = 2 # Добавляем элемент 2   (ключ "b")
```

```
print(d)
{'a': 1, 'b': 2}
```

– с помощью метода dict.fromkeys (<Последовательность> [, <Значение>]). Метод создает новый словарь, ключами которого будут элементы последовательности, переданной первым параметром, а их значениями — величина, переданная вторым параметром. Если второй параметр не указан, то значением элементов словаря будет значение None.

Пример:

```
d = dict.fromkeys( ["a", "b", "c"])
print(d)
{'a': None, 'c': None, 'b': None}
d = dict.fromkeys ( ["a", "b", "c"], 0) # Указан список
print(d)
{'a': 0, 'b': 0, 'c': 0}
d = dict.fromkeys ( ("a", "b", "c"), 0) # Указан кортеж
print(d)
{'a': 0, 'b': 0, 'c': 0}
```

При создании словаря в переменной сохраняется ссылка на объект, а не сам объект. Это обязательно следует учитывать при групповом присваивании. Групповое присваивание можно использовать для чисел и строк, но для списков и словарей этого делать нельзя. **Рассмотрим пример:**

```
d = dict.fromkeys( ["a", "b", "c"])
print(d)
{'a': None, 'c': None, 'b': None}
d = dict.fromkeys ( ["a", "b", "c"], 0) # Указан список
print(d)
{'a': 0, 'b': 0, 'c': 0}
d = dict.fromkeys ( ("a", "b", "c"), 0) # Указан кортеж
print(d)
{'a': 0, 'b': 0, 'c': 0}
```

Как видно из примера, изменение значения в переменной d2 привело также к изменению значения в переменной d1. То есть, обе переменные ссылаются на один и тот же объект, а не на два разных объекта. Чтобы получить два объекта, необходимо производить раздельное присваивание:

```
d1, d2 = {"a": 1, "b": 2}, {"a": 1, "b": 2}
d2["b"] = 10
print(d1, d2)
({'a': 1, 'b': 2}, {'a': 1, 'b': 10})
```

Создать поверхностную копию словаря позволяет функция dict ().

Создание поверхностной копии словаря с помощью функции dict ()

```
d1 = {"a": 1, "b": 2} # Создаем словарь
d2 = dict(d1) # Создаем поверхностную копию
print(d1 is d2) # Оператор показывает, что это разные объекты
False
d2 ["b"] = 10
```

```
print(d1, d2)  # Изменилось только значение в переменной d2
({'a': 1, 'b': 2}, {'a': 1, 'b': 10})
```

Кроме того, для создания поверхностной копии можно воспользоваться методом copy ().

Создание поверхностной копии словаря с помощью метода copy()

```
d1 = {"a": 1, "b": 2} # Создаем словарь
d2 = d1.copy () # Создаем поверхностную копию
print(d1 is d2) # Оператор показывает, что это разные объекты
False
d2 ["b"] = 10
print(d1, d2)  # Изменилось только значение в переменной d2
({'a': 1, 'b': 2}, {'a': 1, 'b': 10})
```

Чтобы создать полную копию словаря, следует воспользоваться функцией deepcopy () из модуля copy.

Создание полной копии словаря

```
d1 = {"a": 1, "b": [20, 30, 40] }
d2 = dict(d1)  # Создаем поверхностную копию
d2["b"][0] = "test"
print(d1, d2) # Изменились значения в двух переменных!!!
({'a': 1, 'b': ['test', 30, 40]}, {'a': 1, 'b': ['test', 30, 40]})
import copy
d3 = copy.deepcopy (d1) # Создаем полную копию
d3["b"][1] = 800
print(d1, d3) # Изменилось значение только в переменной d3
({'a': 1, 'b': ['test', 30, 40]}, {'a': 1, 'b': ['test', 800, 40]})
```

4.3 Операции над словарями в Python

Обращение к элементам словаря осуществляется с помощью квадратных скобок, в которых указывается ключ. В качестве ключа можно указать неизменяемый объект — например: число, строку или кортеж.

Выведем все элементы словаря:

```
d = {1: "int", "a": "str", (1, 2): "tuple"}
print(d[1], d["a"], d[(1, 2)])
('int', 'str', 'tuple')
```

Если элемент, соответствующий указанному ключу, отсутствует в словаре, то возбуждается исключение KeyError:

```
d = {"a": 1, "b": 2}
print(d["c"]) # Обращение к несуществующему элементу
Traceback (most recent call last):
  File "<pyshell#49>", line 1, in <module>
    d["c"] # Обращение к несуществующему элементу
KeyError: 'c'
```

Проверить существование ключа можно с помощью оператора in. Если ключ найден, то возвращается значение True, в противном случае — False.

Примеры:

```
d = {"a": 1, "b": 2}
print("a" in d) #  Ключ существует
True
print("c" in d) #  Ключ не существует
False
```

Проверить, отсутствует ли какой-либо ключ в словаре, позволит оператор not in. Если ключ отсутствует, возвращается True, иначе — False.

Примеры:

```
d = {"a": 1, "b": 2}
print("c" not in d) #  Ключ не существует
True
print("a" not in d) #  Ключ существует
False
```

Метод get (<Ключ>[, <Значение по умолчанию»]) позволяет избежать возбуждения исключения KeyError при отсутствии в словаре указанного ключа. Если ключ присутствует в словаре, то метод возвращает значение, соответствующее этому ключу. Если ключ отсутствует, то возвращается None или значение, указанное во втором параметре.

Пример:

```
d = {"a": 1, "b": 2}
print(d.get ("a") , d.get ("c"), d.get("c", 800))
(1, None, 800)
```

Кроме того, можно воспользоваться методом setdefault (<Ключ> [, <Значение по умолчанию»]). Если ключ присутствует в словаре, то метод возвращает значение, соответствующее этому ключу. Если ключ отсутствует, то в словаре создается новый элемент со значением, указанным во втором параметре. Если второй параметр не указан, значением нового элемента будет None.

Пример:

```
d = {"a": 1, "b": 2}
print(d.setdefault ("a"), d.setdefault ("c"), d.setdefault ("d", 0))
(1, None, 0)
print(d)
{'d': 0, 'a': 1, 'b': 2, 'c': None}
```

Так как словари относятся к изменяемым типам данных, можно изменить элемент по ключу. Если элемент с указанным ключом отсутствует в словаре, то он будет добавлен в словарь:

```
d = {"a": 1, "b": 2}
d["a"] = 800 # Изменение элемента по ключу
d["c"] = "string" # Будет добавлен новый элемент
print(d)
{'a': 800, 'b': 2, 'c': 'string'}
```

Получить количество ключей в словаре позволяет функция len ():

```
d = {"a": 1, "b": 2}
print(len(d)) #  Получаем количество ключей в  словаре
```

```
2
```

Удалить элемент из словаря можно с помощью оператора del:

```
d = {"a": 1, "b": 2}
del d["b"]
print(d)   # Удаляем элемент с ключом "b" и выводим словарь
{'a': 1}
```

4.4 Методы для работы словарями в Python

Для работы со словарями предназначены следующие методы:

– keys () — возвращает объект dict_keys, содержащий все ключи словаря. Этот объект поддерживает итерации, а также операции над множествами.

Пример:

```
d1, d2 = { "a": 1, "b": 2 }, { "a": 3, "c": 4, "d": 5 }
print(d1.keys(), d2.keys()) # Получаем объект dict_keys
(dict_keys(['b', 'a']), dict_keys(['d', 'c', 'a']))
print( list (d1.keys ()), list (d2.keys())) # Получаем список ключей
(['b', 'a'], ['d', 'c', 'a'])
print(for k in d1.keys(): print (k, end=" "))
a b
print(d1.keys() | d2.keys())   # Объединение
{'c', 'd', 'b', 'a'}
print(d1.keys() - d2.keys() )# Разница
{'b'}
print(d2.keys() - d1.keys()) # Разница
{'d', 'c'}
print(d1.keys() & d2.keys()) # Одинаковые ключи
{'a'}
print(d1.keys() ^ d2.keys()) # Уникальные ключи
{'c', 'd', 'b'}
```

– values () — возвращает объект dict_values, содержащий все значения словаря. Этот объект поддерживает итерации.

Примеры:

```
d = { "a": 1, "b": 2 }
print(d.values()) # Получаем объект dict_values
dict_values([2, 1])
print(list (d.values())) # Получаем список значений
[2, 1]
print( [ v for v in d.values() ])
[2, 1]
```

– items () — возвращает объект dict_items, содержащий все ключи и значения в виде кортежей. Этот объект поддерживает итерации.

Примеры:

```
d = { "a": 1, "b": 2 }
print(d.items ()) # Получаем объект dict_items
```

```
dict_items([('b', 2), ('a', 1)])
print(list( d.items () )) # Получаем список кортежей
[('b', 2), ('a', 1)]
```

– <Ключ> in <Словарь> – проверяет существование указанного ключа в словаре. Если ключ найден, то возвращается значение True, в противном случае – False.

Примеры:

```
d = { "a": 1, "b": 2 }
print("a" in d) #  Ключ существует
True
print("c" in d) # Ключ не существует
False
```

– <Ключ> not in <Словарь> – проверяет отсутствие указанного ключа в словаре. Если такового ключа нет, то возвращается значение True, в противном случае – False.

Примеры:

```
d = { "a": 1, "b": 2 }
print("c" not in d) # Ключ не существует
True
print("a" not in d) #  Ключ существует
False
```

– get (<Ключ>[, <Значение по умолчания»]) – если ключ присутствует в словаре, то метод возвращает значение, соответствующее этому ключу. Если ключ отсутствует, то возвращается None или значение, указанное во втором параметре.

Пример:

```
 d = { "a": 1, "b": 2 }
 print(d.get("a") , d.get ("c"), d.get("c", 800) )
 (1, None, 800)
```

– setdefault (<Ключ>[, <Значение по умолчанию>)) – если ключ присутствует в словаре, то метод возвращает значение, соответствующее этому ключу. Если ключ отсутствует, то создает в словаре новый элемент со значением, указанным во втором параметре. Если второй параметр не указан, значением нового элемента будет None.

Пример:

```
d = { "a": 1, "b": 2 }
print(d.setdefault ("a"), d.setdefault ("c") ,d.setdefault ("d", 0))
(1, None, 0)
print(d)
{'c': None, 'd': 0, 'b': 2, 'a': 1}
```

– pop (<ключ> [, <Значение по умолчанию>]) – удаляет элемент с указанным ключом и возвращает его значение. Если ключ отсутствует, то возвращается значение из второго параметра. Если ключ отсутствует, и второй параметр не указан, возбуждается исключение KeyError.

Примеры:

```
d = { "a": 1, "b": 2, "c": 3 }
```

```
print(d.pop("a"), d.pop("n", 0))
(1, 0)
print(d.pop("n")) # Ключ отсутствует и нет второго параметра
Traceback (most recent call last):
  File "<pyshell#24>", line 1, in <module>
    d.pop("n") # Ключ отсутствует и нет второго параметра
KeyError: 'n'
print(d)
{'c': 3, 'b': 2}
```

– popitem () – удаляет произвольный элемент и возвращает кортеж из ключа и значения. Если словарь пустой, возбуждается исключение KeyError.

Примеры:

```
d = { "a": 1, "b": 2 }
print(d.popitem()) # Удаляем произвольный элемент
('b', 2)
print(d.popitem()) # Удаляем произвольный элемент
('a', 1)
print(d.popitem()) # Словарь пустой. Возбуждается исключение
Traceback (most recent call last):
  File "<pyshell#29>", line 1, in <module>
    d.popitem() # Словарь пустой. Возбуждается исключение
KeyError: 'popitem(): dictionary is empty'
```

– clear () – удаляет все элементы словаря. Метод ничего не возвращает в качестве значения.

Пример:

```
d = { "a": 1, "b": 2 }
d.clear() # Удаляем все элементы
print(d)  # Словарь теперь пустой
{}
```

– update () – добавляет элементы в словарь. Метод изменяет текущий словарь и ничего не возвращает. **Форматы метода:**

update(<Ключ1>=<Значение1>[, ..., <КлючN>=<ЗначениеN>])
update(<Словарь>)
update(<Список кортежей с двумя элементами>)
update(<Список списков с двумя элементами>)

Если элемент с указанным ключом уже присутствует в словаре, то его значение будет перезаписано.

Примеры:

```
d = { "a": 1, "b": 2 }
d.update (c=3, d=4)
print(d)
{'c': 3, 'd': 4, 'b': 2, 'a': 1}
d.update({"c": 10, "d": 20}) # Словарь
print(d) # Значения элементов перезаписаны
{'a': 1, 'd': 20, 'b': 2, 'c': 10}
```

```
d.update([ ("d", 80), ("e", 6)]) # Список кортежей
print(d)
{'a': 1, 'd': 80, 'b': 2, 'c': 10, 'e': 6}
d.update ([ ["a", "str"], ["i", "t"] ]) # Список списков
print(d)
{'a': 'str', 'i': 't', 'd': 80, 'b': 2, 'c': 10, 'e': 6}
```

– copy () – создает поверхностную копию словаря:

```
d1 = { "a": 1, "b": [10, 20] }
d2 = d1.copy()  #  Создаем поверхностную копию
print(d1 is d2) #  Это разные объекты
False
d2["a"] = 800 # Изменяем значение
print(d1, d2) #  Изменилось значение только в d2
({'b': [10, 20], '?': 1}, {'a': 800, 'b': [10, 20]})
d2["b"][0] = "new"  #  Изменяем значение вложенного списка
print(d1, d2) # Изменились  значения и в d1, и в d2!!!
({'b': ['new', 20], 'a': 1}, {'a': 800, 'b': ['new', 20]})
```

Чтобы создать полную копию словаря, следует воспользоваться функцией deepcopy () из модуля copy.

4.5 Перебор элементов словаря

Перебрать все элементы словаря можно с помощью цикла **for**, хотя словари и не являются последовательностями. Для примера выведем элементы словаря двумя способами. Первый способ использует метод **keys ()**, возвращающий объект с ключами словаря. Во втором случае мы просто указываем словарь в качестве параметра. На каждой итерации цикла будет возвращаться ключ, с помощью которого внутри цикла можно получить значение, соответствующее этому ключу:

```
d = {"x": 1, "y": 2, "z": 3}
for key in d.keys(): # Использование метода keys()
    print("({0} => {1})".format(key, d[key]), end=" ")
# Выведет: (y => 2) (x => 1) (z => 3)
print()    # Вставляем символ перевода строки
for key in d:       # Словари также поддерживают итерации
    print("({0} => {1})".format(key, d[key]), end=" ")
# Выведет: (y => 2) (x => 1) (z => 3)
```

Поскольку словари являются неупорядоченными структурами, элементы словаря выводятся в произвольном порядке. Чтобы вывести элементы с сортировкой по ключам, следует получить список ключей, а затем воспользоваться методом **sort ()**.

Пример:

```
d = {"x": 1, "y": 2, "z": 3}
k = list(d.keys()) #  Получаем список ключей
k.sort() #  Сортируем список ключей
for key in k:
```

```
    print("({0} => {1})".format(key, d[key]), end=" ")
# Выведет: (x => 1) (y => 2) (z => 3)
```

Для сортировки ключей вместо метода **sort ()** можно воспользоваться функцией **sorted()**. Пример:

```
d = {"x": 1, "y": 2, "z": 3}
for key in sorted(d.keys()):
    print("({0} => {1})".format(key, d[key]), end=" ")
# Выведет: (x => 1) (y => 2) (z => 3)
```

Так как на каждой итерации возвращается ключ словаря, функции **sorted ()** можно сразу передать объект словаря, а не результат выполнения метода **keys()**:

```
d = {"x": 1, "y": 2, "z": 3}
for key in sorted(d):
    print("({0} => {1})".format(key, d[key]), end=" ")
# Выведет: (x => 1) (y => 2) (z => 3)
```

4.6 Генераторы словарей

Помимо генераторов списков, язык **Python 3** поддерживает ***генераторы словарей***. Синтаксис генераторов словарей похож на синтаксис генераторов списков, но имеет два отличия:

- выражение заключается в фигурные скобки, а не в квадратные;
- внутри выражения перед циклом **for** указываются два значения через двоеточие, а не одно. Значение, расположенное слева от двоеточия, становится ключом, а значение, расположенное справа от двоеточия, - значением элемента.

Пример:

```
keys = ["a", "b"] # Список с ключами
values = [1, 2] # Список со значениями
print({k: v for (k, v) in zip(keys, values)})
{'a': 1, 'b': 2}
print({k: 0 for k in keys})
{'a': 0, 'b': 0}
```

Генераторы словарей могут иметь сложную структуру. Например, состоять из нескольких вложенных циклов **for** и (или) содержать оператор ветвления **if** после цикла. Создадим новый словарь, содержащий только элементы с четными значениями, из исходного словаря:

```
d = {"a": 1, "b": 2, "c": 3, "d": 4}
print(k: v for (k, v) in d.items() if v % 2 == 0})
{'d': 4, 'b': 2}
```

4.7 Множества в Python

Множество – это неупорядоченная последовательность уникальных элементов, с которой можно сравнивать другие элементы, чтобы определить, принадлежат ли они этому множеству. Объявить множество можно с помощью функции **set()**:

```
s = set ()
print(s)
set( )
```

Функция **set ()** позволяет также преобразовать элементы последовательности во множество:

```
print(set("string")) # Преобразуем строку
{'s', 't', 'r', 'i', 'n', 'g'}
print(set ([1,2,3,4,5])) # Преобразуем список
{l, 2, 3, 4, 5}
print(set ((1,2,3,4,5))) # Преобразуем кортеж
{l, 2, 3, 4, 5}
print(set([1, 2, 3, 1, 2, 3])) # Остаются только уникальные элементы
{1, 2, 3}
```

Перебрать элементы множества позволяет цикл **for**:

```
for i in set([l, 2, 3]): print(i)
1 2 3
```

Получить количество элементов множества позволяет функция **len()**:

```
print(len(set( [1, 2, 3] ) ))
3
```

4.8 Операторы и методы работы со множествами

Для работы с множествами предназначены следующие операторы и соответствующие им методы:

– | и **union ()** – объединяет два множества:

```
s = set ( [1, 2, 3] )
print(s.union(set ([4, 5, 6])), s | set ([4, 5, 6]))
( {l, 2, 3, 4, 5, 6}, {l, 2, 3, 4, 5, 6} )
```

Если элемент уже содержится во множестве, то он повторно добавлен не будет:

```
print(set([1, 2, 3]) | set([1, 2, 3]))
{l, 2, 3}
```

– **a |= b** и **a.update(b)** – добавляют элементы множества **b** во множество **a**:

```
s = set ( [1, 2, 3] )
s.update (set ( [4, 5, 6] ) )
print(s)
{1, 2, 3, 4, 5, 6}
s |= set ([7, 8, 9])
print(s)
{l, 2, 3, 4, 5, 6, 7, 8, 9}
```

– и **difference()** - вычисляет разницу множеств:

```
print(set([1, 2, 3]) - set([1, 2, 4]))
{3}
s  = set ([1, 2, 3])
print(s.difference(set ( [1, 2, 4] ) ) )
```

```
{3}
```

– **a -= b** и **a.difference_update (b)** – удаляют элементы из множества a, которые существуют и во множестве **a**, и во множестве **b**:

```
s = set ( [1, 2, 3] )
s.difference_update(set ( [1, 2, 4] ) )
print(s)
{3}
s  -= set ( [3, 4, 5] )
print(s)
set( )
```

– **&** и **intersection()** - пересечение множеств. Позволяет получить элементы, которые существуют в обоих множествах:

```
print(set([1, 2, 3]) & set([1, 2, 4]))
{1, 2}
s  = set ([1, 2, 3])
print(s.intersection (set([1, 2, 4])))
{1, 2}
```

– **a &= b** и **a.intersection_update(b)** - во множестве **a** останутся элементы, которые существуют и во множестве **a**, и во множестве **b**:

```
s  =  set ( [1, 2, 3] )
s.intersection_update(set ( [1, 2, 4] ) )
print(s)
{1, 2}
s &= set ([1, 6, 7])
print(s)
{1}
```

– ^ и **symmetric_difference()** - возвращают все элементы обоих множеств, исключая элементы, которые присутствуют в обоих этих множествах:

```
s  = set ( [1, 2, 3] )
print(s  ^ set([l, 2, 4]), s.syinmetric_difference(set ( [1, 2, 4] ) ))
( {3, 4}, {3, 4} )
print(s ^ set( [1, 2, 3] ), s.syiranetric_difference(set( [1, 2, 3] ) ))
( set( ), set( ) )
print(s ^ set( [4, 5, 6]), s.symmetric_difference(set( [4, 5, 6] ) ))
( {1, 2, 3, 4, 5, 6}, {l, 2, 3, 4, 5, 6} )
```

– **a ^= b** и **a.symmetric_difference_update(b)** - во множестве **a** будут все элементы обоих множеств, исключая те, что присутствуют в обоих этих множествах:

```
s = set ( [1, 2, 3] )
s.syrametric_difference_update (set ( [1, 2, 4] ))
print(s)
{3, 4}
s ^= set ( [3, 5, 6] )
print(s)
{4, 5, 6}
```

4.9 Операторы сравнения множеств

Перечислим операторы сравнения множеств.

- **in** - проверка наличия элемента во множестве:

```
s  =  set( [1, 2, 3, 4, 5])
print(1 in s, 12 in s)
(True, False)
```

- **not in** - проверка отсутствия элемента во множестве:

```
s  =  set( [1, 2, 3, 4, 5])
print(1 not in s, 12 not in s)
(False, True)
```

- == - проверка на равенство:

```
print(set( [1, 2, 3]) == set([1, 2, 3]))
True
print(set( [1, 2, 3]) == set([3, 2, 1]))
True
print(set( [1, 2, 3]) == set([l, 2, 3, 4]))
False
```

- **a <= b** и **a.issubset (b)** - проверяют, входят ли все элементы множества **a** во множество **b**:

```
s  =  set( [1, 2, 3])
print(s <= set([l, 2]), s <= set([1, 2, 3, 4]))
(False, True)
print(s.issubset (set( [1, 2])), s.issubset (set( [1, 2, 3, 4])))
(False, True)
```

- **a < b** - проверяет, входят ли все элементы множества **a** во множество **b**, причем множество **a** не должно быть равно множеству **b**:

```
s = set ( [1, 2, 3] )
print(s < set([1, 2, 3]), s < set([1, 2, 3, 4]))
(False, True)
```

- **a >= b** и **a.issuperset (b)** – проверяют, входят ли все элементы множества **b** во множество **a**:

```
s  = set([1, 2, 3] )
print(s >= set([l, 2]), s >= set([1, 2, 3, 4]))
(True, False)
print(s.issuperset (set ([1, 2])), s.issuperset (set ([1, 2, 3, 4] ) ))
(True, False)
```

- **a > b** - проверяет, входят ли все элементы множества **b** во множество **a**, причем множество **a** не должно быть равно множеству **b**:

```
s  = set([1, 2, 3])
print(s > set([1, 2]), s > set([1, 2, 3]))
(True, False)
```

- **a.isdisjoint (b)** - проверяет, являются ли множества **a** и **b** полностью разными, т. е. не содержащими ни одного совпадающего элемента:

```
s =  set([1, 2, 3])
```

```
print(s.isdisjoint (set ( [4, 5, 6] ) ))
True
print(s.isdisjoint (set ([1, 3, 5])))
False
```

4.10 Методы работы со множествами

Для работы с множествами предназначены следующие методы:

- **copy ()** - создает копию множества. Обратите внимание на то, что оператор = присваивает лишь ссылку на тот же объект, а не копирует его.

Пример:

```
s = set ( [1, 2, 3] )
c = s
print(s is c) #С помощью = копию создать нельзя!
True
c = s.copy()  # Создаем копию объекта
print©
{1, 2, 3}
print(s is c) # Теперь это разные объекты
False
```

- **add (<Элемент>)** - добавляет **<Элемент>** во множество:

```
s = set ( [1, 2, 3] )
s.add(4)
print(s)
{1, 2, 3, 4}
```

- **remove (<Элемент>)** - удаляет **<Элемент>** из множества. Если элемент не найден, то возбуждается исключение **KeyError**:

```
s = set ( [1, 2, 3] )
s.remove(3)
print(s) # Элемент существует
{l, 2}
print(s.remove(5)) # Элемент НЕ существует
Traceback (most recent call last);
File "<pyshell#78>", line 1, in <module>
  s.remove(5) # Элемент НЕ существует
KeyError: 5
```

- **discard (<Элемент>)** - удаляет **<Элемент>** из множества, если он присутствует. Если указанный элемент не существует, никакого исключения не возбуждается:

```
s = set ( [1, 2, 3] )
s.discard(3)
print(s) # Элемент существует
{l, 2}
print(s.discard(5)) # Элемент НЕ существует
{l, 2}
```

– **pop ()** - удаляет произвольный элемент из множества и возвращает его. Если элементов нет, то возбуждается исключение **KeyError**:

```
s = set ( [1, 2] )
print(s.pop (), s)
(1, {2})
print(s.pop (), s)
(2, set())
print(s.pop())  #  Если нет элементов, то будет ошибка
Traceback  (most  recent  call  last):
File "<pyshell#89>",  line  1,  in <module>
  s.pop()  #  Если нет элементов, то будет ошибка
KeyError: 'pop from an empty set'
```

– **clear ()** - удаляет все элементы из множества:

```
s = set ( [1, 2, 3] )
s.clear ()
print(s)
set()
```

Помимо генераторов списков и генераторов словарей, язык **Python 3** поддерживает **генераторы множеств**. Синтаксис генераторов множеств похож на синтаксис генераторов списков, но выражение заключается в фигурные скобки, а не в квадратные. Так как результатом является множество, все повторяющиеся элементы будут удалены.

Пример:

```
print({x for x in [1, 2, 1, 2, 1, 2, 3]})
{1, 2, 3}
```

Генераторы множеств могут иметь сложную структуру. Например, состоять из нескольких вложенных циклов **for** и (или) содержать оператор ветвления **if** после цикла. Создадим из элементов исходного списка множество, содержащее только уникальные элементы с четными значениями:

```
print({x for x in [1, 2, 1, 2, 1, 2, 3] if x % 2 == 0})
{2}
```

Язык **Python** поддерживает еще один тип множеств - **frozenset**. В отличие от типа **set**, множество типа **frozenset** нельзя изменить. Объявить множество можно с помощью функции **frozenset()**:

```
f = frozenset()
print(f)
```

Функция frozenset() позволяет также преобразовать элементы последовательности во множество:

```
f = frozenset("string") #  Преобразуем строку
print(f)
frozenset({'n', 's', 'r', 'g', 't', 'i'})
f = frozenset( [1, 2, 3, 4, 4] )  # Преобразуем список
print(f)
frozenset({1, 2, 3, 4})
f = frozenset( (1, 2, 3, 4, 4) ) # Преобразуем кортеж
```

```
print(f)
frozenset({1, 2, 3, 4})
```

Множества **frozenset** поддерживают операторы, которые не изменяют само множество, а также следующие методы: **copy()**, **difference()**, **intersection()**, **issubset()**, **issuperset()**, **symmetric_difference()** и **union()**.

4.11 Пример решения задачи на языке программирования Python

Задача. Разработайте программу, которая выводит фразу «Привет, программист» на одном из иностранных языков.

Комментарий. При решении данной задачи мы создадим словарь из нескольких пар, в которых есть ключ – страна и значение, которым является приветствие. Все имеющиеся пары мы заключаем в фигурные скобки. Далее с помощью условного оператора **if** проверяем, соответствует ли введенное название страны ключу в словаре и, если условие оказывается истинным, выводим соответствующее значение.

Решение задачи:

```
slovar={"Россия":"Привет, программист!",
        "Англия":"Hello, programmer",
        "Германия":"Hallo, Programmierer",
        "Испания":"Hola, programador",
        "Италия": "Ciao, programmatori"}
slovo=input("\nВведите название страны ")
if slovo in slovar:
   perevod=slovar[slovo]
   print("\n", slovo, " Приветствие звучит как ", perevod)
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте словарь d = {1: 2, 3: 4, 4: 3, 2: 1, 0: 0} и отсортируйте его по значениям в порядке возрастания и убывания.

2 Создайте словари dict_a = {1:10, 2:20}, dict_b = {3:30, 4:40} и dict_c = {5:50, 6:60}, после чего объедините их в один словарь.

3 Создайте словарь my_dict = {'a':500, 'b':5874, 'c':560, 'd':400, 'e':5874, 'f':20} и найдите три ключа, соответствующие наибольшим значениям.

4 Создайте словари dictionary_1 = {'a':300, 'b':400} и dictionary_2 = {'c':500, 'd':600}, затем объедините их с использованием встроенных средств языка Python.

5 Создайте словарь, в котором ключами являются числа от 1 до 10, а значениями — кубы этих чисел.

6 Создайте словарь по двум спискам одинаковой длины так, чтобы элементы первого списка являлись ключами, а элементы второго — значениями.

7 Создайте словарь по строке 'pythonist', где ключами являются символы строки, а значениями — количество их вхождений.

8 Создайте словарные представления двух комплексных чисел и выполните их сложение и умножение.

9 Создайте данные для двух дат (месяц и год) и определите, принадлежит ли введённая пользователем дата заданному диапазону включительно.

10 Создайте словарь, содержащий числа от 1 до N в качестве ключей и их квадраты в качестве значений.

11 Создайте словарь по двум введённым спискам, где ключами являются элементы первого списка, а значениями — соответствующие элементы второго списка.

12 Создайте словарь частот встречаемости символов для заданного текста.

13 Создайте словарь по заданному списку чисел, в котором ключами являются элементы списка, а значениями — количество их вхождений.

14 Создайте словарь и определите ключ, которому соответствует максимальное значение.

15 Создайте новый словарь на основе заданного словаря, включив в него только те элементы, значения которых превышают указанное пользователем число.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте два множества с различными элементами и выведите их объединение.

2 Создайте два множества с некоторыми общими элементами и выведите их пересечение.

3 Создайте два множества и найдите разницу между ними (элементы, которые есть в первом множестве, но отсутствуют во втором).

4 Создайте два множества и проверьте, является ли одно из них подмножеством другого.

5 Создайте два множества с непересекающимися элементами и проверьте, являются ли они непересекающимися множествами.

6 Создайте список с повторяющимися элементами и преобразуйте его в множество, чтобы получить уникальные значения.

7 Создайте список с дубликатами и удалите их, преобразовав список во множество и обратно.

8 Создайте два множества и проверьте, имеют ли они хотя бы один общий элемент.

9 Создайте два множества и проверьте, являются ли они дизъюнктивными (не имеют общих элементов).

10 Создайте множество из строки и выведите все уникальные символы этой строки.

11 Создайте множество из списка и удалите все дубликаты из списка.

12 Создайте два множества и проверьте, является ли одно из них подмножеством другого или дополнительным множеством.

13 Создайте множество с буквами алфавита и найдите объединение с множеством гласных букв.

14 Создайте множество целых чисел и определите количество его элементов, значения которых являются четными.

15 Создайте два множества чисел и сформируйте новое множество, содержащее только те элементы, которые присутствуют в одном из множеств и отсутствуют в другом.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Дайте определение понятию «Словарь» в языке программирования Python.

7.2 Приведите синтаксис объявления словарей в языке программирования Python.

7.3 Перечислите и опишите методы для работы со словарями в языке программирования Python.

7.4 Перечислите и опишите операторы и методы работы со множествами в языке Python.

7.5 Перечислите и опишите операторы сравнения множеств в языке программирования Python.

7.6 Перечислите и опишите методы работы со множествами в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.